

ZeusChain: Interchain Money and an Interchain World

Wei Wang
Providence High, Charlotte, NC 28270

October 1, 2024

Abstract

Secure interoperability hubs are crucial for the success of a multi-chain future. Over the past decade, numerous blockchains have been developed, each supporting large user communities, developers, and millions of digital assets. Various scaling solutions have been implemented to achieve the speed and efficiency of traditional systems like credit card networks. However, with the rise of new Layer 2 and Layer 1 solutions, the industry faces a significant challenge: ensuring the security of cross-chain protocols. Most mainstream Byzantine Fault Tolerance (BFT) consensus mechanisms at Internet scale do not meet the rigorous security standards set by academic research. To address this, we developed a novel consensus mechanism specifically optimized for cross-chain security. We introduce a comprehensive blockchain ecosystem featuring an integrated Layer 1 ZeusChain and a stablecoin economy, designed to connect various blockchains, their communities, and assets in a decentralized manner without relying on trusted organizations. ZeusChain aims to be the most secure system for enabling trustless transactions across existing chains. This paper presents the characteristics and inter-chain communication mechanisms of ZeusChain, followed by an exploration of the network's broader purpose and goals. The prototype implementation of our ZeusChain is available at our GitHub site (the full address will be provided after the paper is accepted for publication).

1 Introduction

In this paper, we present ZeusChain, a novel blockchain network architecture that allows for multiple parallel blockchains to interoperate. We take a layered approach; at the base layer, we use the BDLS protocol from Wang [?]. The BDLS BFT consensus mechanism is optimized for scaling public proof-of-stake blockchains, while offering greater security and efficiency when compared to other Byzantine Fault Tolerant protocols. On top of it, we introduce ZeusChain, an interoperability hub that contains several parallel co-chains, or “Statechains.” Rather than building L2s to singular blockchains as is most common today, this architecture allows ZeusChain to serve as Layer 2s to all major blockchains. Both the ZeusChain and its Statechains utilize the BDLS mechanism. At the application layer, “ZeusChain 1” is a full-stack DeFi ecosystem. ZeusChain 1’s economy is powered by a fleet of stablecoins that are algorithmic and ideally-collateralized. Liquidity within ZeusChain 1 can be extended by connecting to other Statechains.

The ZeusChain and Statechains of the ZeusChain network utilize a native inter-blockchain communication protocol. The network includes a built-in asset-swap protocol for seamlessly and quickly exchanging tokens between connected networks. All inter-Statechain token transfers go through the ZeusChain, which keeps track of the total amount of tokens held by each. This interoperable network aims to increase the rate of innovation at both the infrastructure layer as well as the application layer, specifically encouraging experimentation that is traditionally locked out of expensive alternative blockchains. Our guiding thesis is that composability is the key for rapid technological progress.

Compared to credit card payment systems, most of the recent blockchains that have emerged over the years have very small transaction capacity (transaction per second: tps). For example, Bitcoin has approximately 5tps, Ethereum has approximately 15tps, and Visa has approximately 2000tps. In order to increase transactions per second, the Ethereum community has developed various layer 2 techniques that include: state channels, Plasma, rollups, and validium (e.g., Starkware, zkPoter, and Loopring). Some of these Layer 2 techniques are displayed as a two-by-two matrix in Table ??.

Table 1: Ethereum layer 2 two-by-two matrix

	SNARKs/STARKs	Fraud proofs
Data on-chain	ZK rollup	optimistic rollup
Data off-chain	Validium	Plasma

In the Plasma technique, the layer 2 chain only broadcasts cryptographic commitments on the main chain and data verification is carried out on the main chain. In case that incorrect data is committed, one can challenge and revert the commitments. Polygon is an example implementation of the Plasma techniques. In the Polygon system, a user deposits a cryptographic asset in the Matic contract. When the deposit is confirmed, corresponding ERC-20 tokens will be minted on the Matic Chain using the Matic Deposit bridge. Users can withdraw the corresponding Polygon ERC-20 tokens to the main Ethereum chain by establishing proof of remaining tokens on the Root contract. For the Plasma system, a user needs to constantly monitor the root contract to make sure that the committed transaction is correct. In case of a dispute, the layer 2 chain may refuse to release the actual transaction data. A better solution would be to batch many transactions into a single transaction and the validity of these batched transactions is verified using a single protocol such as ZK-STARK proof. This kind of solution is represented by the ZK rollup techniques. Vitalik Buterin showed that 15 bytes could be used to represent a transaction in the batched transaction and each transaction costs around 892 gas. Currently, the cost of a ZK-STARK verification is around 600,000 gas. The gas limit for an Ethereum block is 8M. Thus, if an entire Ethereum block is used for this batched transaction, a single Ethereum block may hold $(8000000 - 600000 - 50000) \div 892 \simeq 8239$ transactions. This means a capacity of 550 transactions per second. On the other hand, it is very expensive to generate a ZK-STARK proof (e.g., it may take more than 20 minutes for a powerful computer). In the optimistic rollup approach, the aggregator deploys a smart contract on the main chain and publishes the bare minimum information needed with no proofs for transactions. The optimistic rollup technique assumes that the aggregators are honest and do not commit frauds, and proofs are needed only if a fraud is reported. The Validium approach also uses ZK-STARK proof for transaction validity but the transaction data is not stored on the main chain. It is estimated that the Validium chains can have 10k tps per chain.

In order to scale Ethereum blockchain, the Ethereum community released the Ethereum 2.0 Specification. The Ethereum 2.0 blockchain consists of 64 shard chains and one beacon chain. The beacon chain coordinates the expanded network of shards and stakers. The 64 shard chain states are periodically recorded on the beacon chain. The governance structures of all Ethereum 2.0 chains are based on Proof of Stakes (PoS) instead of Proof of Work (PoW). To participate in the blockchain governance, a user needs to send 32 ETH to the deposit contract on the beacon chain and the beacon chain assigns stakers to validate shard chains.

There is one proposing validator and 111 attesting validators for each shard chain. Every 6 minutes, a validator is assigned a duty and is rewarded if it performs its duty timely. The PoS consensus protocol requires at least 2/3 honest validators to sign each block. If more than 1/3 of the assigned validators are offline or not participate in the block generation process, the block finality cannot be reached. If this continues for over 4 epochs (25 minutes), all offline validators will incur finality leak penalties.

It is noted that there are quite some challenges for these proposed Ethereum scalability solutions. For example, it is extremely slow to generate ZK-STARK proofs for ZK rollup solutions and one may mount Denial of Service (DoS) attacks against validators in the Ethereum 2.0 solution (this could be achieved since the validator’s identities are known in advance). Furthermore, the process for imposing finality leak penalties against off-line validators could be a single point of failure. Based on our analysis on existing techniques, we propose an innovative ZeusChain architecture in this paper. Instead of using expensive ZK rollups, ZeusChain leverages more efficient Non-Interactive membership Proof systems to achieve batched transaction rollup to the main-chain. Though ZeusChain works as an independent blockchain and it contains all blockchain functionalities, it could also be considered as a layer-2 chain for all other blockchains such as Ethereum, BSC chains, etc. Based on the built-in hub functions and automated market maker functions in the ZeusChain, one can conveniently swap digital properties among different chains in a fully decentralized way.

2 Challenges of existing PoS-based blockchains

The community believes that PoS based blockchains are the future for blockchain technologies. The core technology for blockchains (including both PoW and PoS chains) is the consensus protocol. The consensus protocol determines how to generate the next block. In other words, the consensus protocol establishes a global atomic ordering for all transactions that happen in the system. If we cannot agree on the order of two transactions, then we cannot have trust in the blockchain system. If a malicious attacker could manage to attack this atomic ordering process, then disaster could happen. One of the typical attacks is “double spending”. This kind of attack has already happened in blockchains such as Zencash, Ethereum Classic, and Bitcoin Gold. Generally, a secure blockchain should satisfy the following two conditions:

1. Safety: honest participants should agree on the order of valid transactions and no forks should happen.
2. Liveness: a valid transaction should be included in the blockchain in a reasonable time period.

For a PoS based blockchain, participants satisfying certain conditions could be invited to generate a candidate next block. Normally there are a few participants that satisfy these conditions. Since it is profitable for a participant to generate blocks in the blockchain, each participant would try his/her best to convince the community to accept his/her block as the next block. If two different blocks are accepted as the next block by the community at the same time, a fork is created and the safety property is no longer satisfied. In order to prevent fork from happening, most PoS based blockchains use Byzantine Fault Tolerance (BFT) protocols as a finality gadget to select a unique next-block from a few candidate next-blocks. There are a few challenges for this process to work correctly.

2.1 Existing BFT protocols for PoS blockchains are not sufficient to guarantee blockchain security

As we have mentioned in the preceding paragraphs, normally there are a few participants who are qualified to produce the next candidate block. BFT consensus protocols are frequently used as a finality gadget to select a unique next-block from a few candidate next-blocks. However, a recent analysis by [?, ?] shows that several widely deployed BFT based PoS blockchains may be insecure.

Lamport, Shostak, and Pease [?] and Pease, Shostak, and Lamport [?] initiated the study of reaching consensus in face of Byzantine failures and designed the first synchronous solution for Byzantine agreement. Dolev and Strong [?] proposed an improved protocol in a synchronous network with $O(n^3)$ communication complexity. By assuming the existence of digital signature schemes and a public-key infrastructure, Katz and Koo [?] proposed an expected constant-round BFT protocol in a synchronous network setting against $\lfloor \frac{n-1}{2} \rfloor$ Byzantine faults.

For an asynchronous network, Fischer, Lynch, and Paterson [?] showed that there is no deterministic protocol for the BFT problem in face of a single failure. Several researchers have tried to design BFT consensus protocols to circumvent the impossibility. For example, Ben-Or [?] initiated the probabilistic approach to BFT consensus protocols in completely asynchronous networks and Dwork, Lynch, and Stockmeyer [?] designed BFT consensus protocols in partial synchronous networks. Castro and Liskov [?] initiated the study of practical BFT consensus protocol design and introduced the PBFT protocol for partial synchronous networks. The recent paper [?, ?] contains a security analysis for several PoS based blockchains that are based on the following BFT consensus protocols: Polkadot’s GRANDPA BFT, Ethereum’s Casper the Friendly Finality Gadget, and Ethereum’s CBC, and “Correct-by-Construction” (CBC) family of Casper blockchain consensus protocols for the future release of Ethereum blockchain. In the ZeusChain blockchain, we will use the efficient and mathematically provable secure BDLS BFT consensus protocol. For more details on secure BFT protocols in partially synchronous networks and on the mathematical proofs for BDLS protocols, the reader is referred to Wang [?].

2.2 Achieving unpredictability of next-block-proposer identity

It is very important to guarantee unpredictability of the identity of the next candidate block-producer. If his/her identity is predictable, then there are many potential attacks. As an example, we can describe two

simple scenarios. In the first scenario, a malicious user may bribe the next candidate block producer to exclude certain transactions within the next block. In the second scenario, assuming that both A and B are the next candidate block producers. If A knows the identity of B , A may try to DoS B 's network so that B 's candidate block could not be broadcast to the network. Thus A 's candidate block will be accepted as the final next-block. Several attacks based on the predictability of the next block producer identity have happened already. For example, this has happened several times on the EOSIO Blockchain Software architecture. Many currently deployed PoS blockchains do not address this challenge at all. For example, in the PoS Ethereum 2.0, the beacon chain will assign validators for each sharding chain. These validators will serve the sharding chain for a certain amount of epochs (which is quite long). If an adversary has sufficient funding, the adversary may bribe these validators so that these validators will not function (pretending to have network problems). This will essentially deadlock the targeted sharding chain. Several other blockchains leverage the Verifiable Random Function (VRF) techniques to address this challenge. In the ZeusChain blockchain, these challenges are addressed using an innovative random beacon protocol to guarantee the unpredictability of the next-block producer identity.

3 ZeusChain BFT consensus protocol

ZeusChain is a Proof of Stake (PoS) blockchain based on the efficient BDLS Byzantine Fault Tolerance protocol [?, ?]. In the following, we briefly review the BDLS Byzantine Fault Tolerance consensus protocol. For detailed discussion and security analysis, the reader is referred to [?, ?]. The GO-language implementation of the BDLS BFT protocol is available at <https://labs.hyperledger.org/labs/bdls.html>.

For each blockchain height h , BDLS protocol runs from round to round until it reaches an agreement for the height h . Then the protocol moves to the next blockchain height $h + 1$. Let $P_0, \dots, P_{n-1}$ be the $n = 3t + 1$ participants of the protocol. Assume that there are n_0 valid candidate proposals $B_0^h \prec B_1^h \prec \dots \prec B_{n_0-1}^h$ for the block B^h . During the protocol run, each participant P_i maintains a local variable $\text{BLOCK}_i \subseteq \{B_0^h, B_1^h, \dots, B_{n_0-1}^h\}$ that contains the candidate blocks that it has learned so far. Participant P_i prefers the maximal block in BLOCK_i to be selected as the final block for B^h . The goal of the BDLS protocol is for participants $P_0, \dots, P_{n-1}$ to reach a consensus on the finalized block B^h .

Generally, we can use a robust threshold signature scheme to achieve linear authenticator complexity. For simplicity, the following protocol description is based on a standard digital signature scheme. It could be easily revised to used a threshold signature scheme. Following Dwork, Lynch, and Stockmeyer [?], we assume that all messages after the unknown GST (Global Stabilization Time) is delivered in the same round with any order chosen by the adversary and messages before round GST could get lost. Furthermore, though all participants have a common numbering for the round, they do not know when the round GST occurs. A candidate block B' is acceptable to P_i if P_i does not have a lock on any value except possibly B' . There is a public function $\text{leader}(h, r)$ that returns the round leader for a given round r of the height h . For each height h , the BDLS protocol proceeds from round to round (starting from round 0) until the participant decides on a value. The round r of the height h starts when at least $2t + 1$ participants submit a round-change message to the leader participant. The round r proceeds as follows where $P_i = \text{leader}(h, r)$ is the leader for round r :

1. Each participant P_j (including P_i) sends the signed message $(\langle h, r \rangle_j, \langle h, r, B'_j \rangle_j)$ to the leader P_i where $B'_j \in \text{BLOCK}_j$ is the maximal acceptable candidate block for P_j . The message $\langle h, r \rangle_j$ is considered as a round-change message. After sending the round-change message, P_j will not accept messages for round $r' < r$ anymore except a “decide” message from a previous round.
2. If P_i receives at least $2t + 1$ round-change messages (including himself), it enters round r . In these round-change messages, if there are at least $2t + 1$ signed messages from $2t + 1$ participants with the same candidate block $B' \neq \text{NULL}$, then P_i broadcasts the following signed message (??) to all participants

$$\langle \text{lock}, h, r, B', \text{proof} \rangle_i \tag{1}$$

where proof is a list of at least $2t + 1$ signed messages showing that B' is the candidate blocks for at least $2t + 1$ participants (the proof also shows that round-change request has been authorized by at least $2t + 1$ participants). If P_i does not receive such a block B' , then P_i adds all received candidate blocks

to its local variable BLOCK_i and broadcasts $\langle \text{select}, h, r, B'', \text{proof} \rangle$ where B'' is the candidate block $B'' = \max\{B : B \in \text{BLOCK}_i\}$ and proof is a list of at least $2t + 1$ round-change messages. It should be noted that in order to achieve linear communication complexity when a threshold signature scheme employed, the “proof” in the **lock**-message and **select**-message are different: In the **lock**-message, the “proof” contains an assembled digital signature on the message $\langle h, r, B' \rangle$ while, in the **select**-message, the “proof” contains an assembled digital signature on the message $\langle h, r \rangle$. See Remark 3 for details.

3. If a participant P_j (including P_i) receives a valid $\langle \text{select}, h, r, B'', \text{proof} \rangle$ from P_i during Step ??, then it adds B'' to its BLOCK_j . If a participant P_j (including P_i) receives a valid message $\langle \text{lock}, h, r, B', \text{proof} \rangle_i$ from P_i in Step ??, then it does the following:

- (a) releases any potential lock on B' from previous round, but does not release locks on any other potential candidate blocks
- (b) locks the candidate block B' by recording the valid lock (??)
- (c) sends the following signed commit message to the leader P_i .

$$\langle \text{commit}, h, r, B' \rangle_j. \quad (2)$$

4. If P_i receives at least $2t + 1$ commit messages (??), then P_i decides on the value B' and broadcast the following decide message to all participants

$$\langle \text{decide}, h, r, B', \text{proof} \rangle_i. \quad (3)$$

where proof is a list of at least $2t + 1$ commit messages (??).

5. If a participant P_j (including P_i) receives a decide message (??) from Step 4 or from its neighbor, it decides on the block B' for B^h and moves to the next height $h + 1$ (that is, run the Step ?? of height $h + 1$ by sending the round-change message). Otherwise, it goes to the following lock-release step:
 - (*lock-release*) If a participant P_j (including P_i) has some locked values, it broadcasts all of its locked values with proofs. A participant releases its lock on a value $\langle \text{lock}, h, r'', B'', \text{proof} \rangle_{i''}$ if it receives a lock $\langle \text{lock}, h, r', B', \text{proof} \rangle_{i'}$ with $r' \geq r''$ and $B' \neq B''$.
 - Move to the next round $r + 1$ (i.e., run the Step ?? of height h with $r + 1$).
6. *height synchronization*: At any time during the protocol, if P_j receives a finalized block of height h (e.g., a decide message (??)), P_j decides for height h and moves to height $h + 1$.
7. *round synchronization*: At any time during the protocol, if P_j receives a valid “lock” or “select” or “decide” message for a round $r' > r$, P_j moves to round r' and processes the “lock” or “select” or “decide” message.
8. *timeout*: For each step, P_j should set an appropriate timeout counter. If P_j does not receive enough messages to move forward before timeout counter expires, it moves to the next step.

4 ZeusChain Layer 1 blockchain and its algorithms

In this section, we present ZeusChain’s innovative layer one design that address the major challenges that we have discussed in the preceding section.

4.1 ZeusChain blockchain

ZeusChain proposes a consensus mechanism to simultaneously achieve decentralization, performance and security. The initial status of the block chain is

$$S^0 = \{(P_1, a_1), \dots, (P_j, a_j)\}$$

where $P_1, P_2, \dots, P_j$ are a list of initial users and $a_1, \dots, a_j$ are their respective initial amounts of money units. We assume that each user P_i is identified by its public key pk_i . That is, for the users $P_1, P_2, \dots, P_j$, their corresponding public keys are $pk_1, \dots, pk_j$. In practical implementations, a user P_i may be identified by the hash of her public key. That is, we may use $P_i = H(pk_i)$ in implementations. A valid transaction from a user P_i to a user $P_{i'}$ is in the format of

$$SIG_{pk_i}(P_i, P_{i'}, a')$$

where the user P_i currently has $a \geq a'$ money units, $P_{i'}$ is an existing or a newly created user, and pk_i is the public key of user P_i . The impact of this transaction is that the amount of money units for user P_i is decreased by a' and the amount of money units for user $P_{i'}$ is increased by a' .

In an idealized magic ledger system, all transactions are valid and the list L of sets of transactions are posted in a tamper-proof box in the sky which is visible to all participants

$$L = TX^0, TX^1, TX^2, \dots,$$

ZeusChain block chain is organized in a series of rounds $r = 0, 1, 2, 3, \dots$. Similar to the initial status, the system status for round $r > 0$ is a list of users and their corresponding money units

$$S^r = \left\{ (P_1, a_1^{(r)}), (P_2, a_2^{(r)}), (P_3, a_3^{(r)}), \dots \right\}$$

In a round r , the system status transitions from S^r to S^{r+1} via the transaction set TX^r

$$TX^r : S^r \rightarrow S^{r+1}.$$

ZeusChain blockchain is a list of blocks $B^0, B^1, \dots, B^r$ where each B^r consists of the following fields: the block number r itself, the time-stamp t_r that the block B_r is generated, the set TX^r of transactions for round r , the hash of the previous block $H(B^{r-1})$, the user P^r who generates this block, and a set $CERT^r$ of signatures certifying that the block B^r is constructed appropriately

$$B^r = \{r, t_r, TX^r, H(B^{r-1}), P^r, CERT^r\}.$$

The field $CERT^r$ is a list of signatures for the value $H(r, t_r, TX^r, H(B^{r-1}), P^r)$ from more than $2/3$ of the members of the verifier committee V^r for round r .

In the ZeusChain blockchain, we assume that the network is a Type II partial synchronous network. Specifically, we assume that, at the start of round r , most honest users should have learned the current blockchain $B^0, B^1, \dots, B^{r-1}$. From this chain, one can deduce the user sets $U^0, U^1, \dots, U^{r-1}$ and their corresponding money units of each round. In order for a user to serve as a potential leader or as a verifier committee member, she must have joined the system at least κ -block before where κ is a system parameter. Normally we can take $\kappa = 10$. The choice of $\kappa > 0$ will guarantee that the system is running in a consistent way and also will defeat adaptive key selection attacks (where the adversary spawns new participants with appropriately chosen public keys to increase the chance that these newly spawned participants have a high probability to be chosen as the block proposer).

4.2 ZeusChain random beacon protocol

In the ZeusChain blockchain, a participant is selected to serve as the candidate next-block proposer or as a BFT committee member based on a few random entropies. From a high level, the entropy is from three sources: a random string that the participant would commit in a previous round, randomness from the blockchain itself, and a random string that is based on a coin-flipping protocol such as Randao. The first random source guarantees that the participant identity is not revealed until the participant publishes his/her proposed next candidate block. The third random source guarantees that the process is decentralized, multi-participatory, conspiracy-resistant, and verifiably fair.

ZeusChain's pseudorandom generator leverages the commit-reveal and BLS schemes proposed in the randao protocol [?]. However, the randao protocol is only a public coin-flip protocol and is not sufficient

to protect the candidate next-block proposer's identity before the candidate block is published. ZeusChain blockchain achieves the block proposer' identity protection via a further commit-reveal process. ZeusChain blockchain 1.0 will implement the commit-reveal protocol for the common coin while ZeusChain 2.0 will implement BLS based protocol as specified in [?] for the common coin. In the commit-reveal based common coin, the participant is required to publish his/her random string after the commitment, if he/she does not publish his/her committed string, his/her stake would be slashed. However, if a single participant chooses not to post the committed string, the common coin production process is considered failed and a new commitment process is required. Thus a malicious adversary may try to attack the system by sacrificing his/her tokens. This challenge will be addressed by using the BLS based common coin protocol in ZeusChain 2.0 since the BLS threshold signatures are generated by a group of people, no individual can predict the results, and a single participant cannot stop the post of signature.

For ZeusChain blockchain, if a participant P_i wants to participate in some rounds between $r + \kappa' + 1$ and $r + \kappa' + \kappa''$ (where $\kappa' < \kappa$) as a potential block-proposer or as a potential verifier, P_i chooses a random string γ_i and computes $\kappa'' + 1$ random sequences $\gamma_i^{r+\kappa'} = H(\gamma_i^{r+\kappa'+1})$, $\gamma_i^{r+\kappa'+1+1} = H(\gamma_i^{r+\kappa'+2})$, $\dots$, $\gamma_i^{r+\kappa'+\kappa''} = H(\gamma_i)$. Then the participant P_i submits a transaction

$$SIG_{pk_i} \left(P_i, P_\infty, m_i, \gamma_i^{r+\kappa'}, r + \kappa' \right)$$

to be included in block B^r , where P_∞ is a virtual users for accepting random sequence commitment and $m_i = 10,000a_i$ is the amount of tokens to be staked ($a_i \geq 1$ is an integer).

In ZeusChain 2.0, the common coin $coin_{r+\kappa'}$ for rounds from $r + \kappa' + 1$ to $r + \kappa' + \kappa''$ will be derived using the randao BLS common coin protocol. The details for the randao BLS common coin protocol is referred to [?].

4.3 ZeusChain blockchain parameters

In the following discussions, we will use the following sample parameters:

- $\alpha = 100000$: this is the staking unit (i.e., the number of ZeusChain tokens).
- $\kappa = 20$: if a node wants to participate in the blockchain governance (that is, to become a potential proposer or be part of a BFT committee), the node must submit the staking tokens and its random numbers in advance of κ blocks.
- $E_1 = 5$: the expected value of potential proposers.
- $E_2 = 50$: the expected number of BFT committee size.
- $\gamma = 0$: each staking requires γ ZeusChain tokens. Furthermore, each time when a block is generated, γ ZeusChain tokens will be awarded to the non-default block generator.
- $\gamma' = 1000$: if a node submit an illegal redeem request, γ' ZeusChain tokens will be deducted.
- $\beta_1 = 1000$: the number of ZeusChain tokens awarded to a non-default block proposer.
- $\beta_2 = 3000$: the number of ZeusChain tokens awarded to BFT committee.
- $\xi = 50\%$: the percent of gas fees awarded to the block proposer.
- $\text{MaxUnit256} = 2^{256} - 1$
- There are five default accounts Q_1, Q_2, Q_3, Q_4, Q_5 . Among them, Q_1 is the default potential proposer for each block. Q_2, Q_3, Q_4, Q_5 are default BFT committee members. In the consensus algorithm (??) and (??), the nodes Q_1, Q_2, Q_3, Q_4, Q_5 has the values γ^r and μ_i^r with the following default values: 0x53706572617844656661756c744e6f6465.

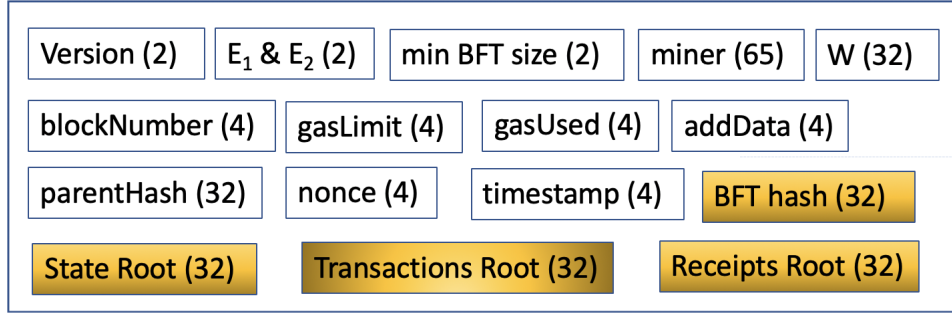


Figure 1: ZeusChain blockchain header

4.4 ZeusChain blockchain header

ZeusChain blockchain header consists of 287 Bytes as shown in Figure ??, where the shared random number W (32 Bytes) are computed as follows:

1. $W_0 = H(0x03243F6A8885A308D313198A2E037073)$.
2. $W_j = H(P^{j-1}, W_{j-1})$ for $0 < j \leq r$, where $P^0 = \emptyset$ and P^j if the proposer of block B^{j-1} .

Since the shared random number W could be computed recursively, we may omit W in the block header.

4.5 Merkle Patricia trie, State Root, World state, and transactions

ZeusChain chain adopts Ethereum compatible technologies. On the other hand, there are some changes in ZeusChain chain and Ethereum codes may not run on ZeusChain chain. In ZeusChain chain head, “State Root(32)” is Merkle Patricia trie’s root hash and it is referred to <https://eth.wiki/en/fundamentals/patricia-tree> for details.

As in the Ethereum blockchain, ZeusChain chain has two kinds of accounts: Externally owned accounts (EOAs) and Contract accounts. Each account contains five fields. The first four fields are the same as Ethereum (see <https://ethereum.github.io/yellowpaper/paper.pdf> for details):

1. nonce: the number of transactions this account has carried out.
2. balance: the balance of ZeusChain tokens.
3. storageRoot: root hash of Merkle Patricia Trie.
4. codeHash: hash of the account contract codes. For EOA account, $\text{codeHash} = H(\emptyset)$
5. stakingTx: the hash of all active staking transactions. An account can have at most two staking transaction

ZeusChain cain has two kinds of transactions: regular transaction and staking transaction. The regular transaction has a similar structure as Ethereum where we replace Ethereum’s EVM codes with BVM codes. ZeusChain chain’s staking transaction includes the following fields:

1. nonce: 0x00 represents staking transactions; 0xFF represents redeeming transaction s
2. gasPrice= γ
3. value= a_i : represents staking (or redeeming) $a_i \alpha$ ZeusChain
4. randomCommitRoot: random number commit
5. startBlock: the starting block height that the node wants to participate in the chain governance.
6. duration: the total numbers of block heights that the node will participate in the chain governance.
7. signature = (v, r, s)

4.6 Active Staking Transaction (ASTX)

Similar to Bitcoin's UTXO (Unspent Transaction Output), BLDS chain uses STX (Active Staking Transaction) to speed up the process of active staking transactions. ASTX manages the expired or redeemed staking transaction in a linear order based on the time ¹. This will help the nodes to process all staking transactions. For example, this will speed up the calculation of the total number of staking ZeusChain tokens for each block and the deletion of expiring staking transactions from the world state. On the other hand, if one wants to calculate the number of ZeusChain tokens that a specific account has on a specific block height, it should be more efficient to use world state. The reader is referred to <https://eprint.iacr.org/2017/1095.pdf> for Bitcoin's UTXO techniques.

4.7 Generating a block B^r in ZeusChain chain

Assuming that the blocks $B^0, B^1, \dots, B^{r-1}$ ($r \geq \kappa$) have been confirmed by the BFT committee. and the next block B^r needs to be proposed. The node P_i enters the following steps for the block of height r .

1. $P_i \neq Q_j$ distinguishes the following cases:
 - (a) If P_i neither wants to submit a staking (stake for a height $r' > r + \kappa$) nor want to redeem transaction: go to Step ??.
 - (b) If P_i wants to redeem staking or stake for a height $r' > r + \kappa$: goto Step ??. Note that if a node wants to redeem or have a plan to stake, the node can prepare the calculation for the Step ??.
2. P_i distinguishes the following cases:
 - (a) $P_i \neq Q_j$ has not staked for height r : P_i waits for the BFT committee to confirm the block B^r . After P_i receives the confirmed block B^r , it goes to Step 1 of height $r + 1$.
 - (b) P_i has staked $a_i \alpha$ ZeusChain tokens for height r or $P_i = Q_j$: First, P_i calculates the following value:
 - Calculates the total number Ω_r of ZeusChain tokens staked for the height r (see Section ??).
 - Calculate the probability $p = \frac{E_1 \alpha}{\Omega_r}$ and $p' = \frac{E_2 \alpha}{\Omega_r}$.
 - Caculate the shared random number W_r for the height r .

If $P_i = Q_1$, then $P_i = Q_1$ is a potential proposer. If $P_i = Q_2, Q_3, Q_4, Q_5$, then P_i is a BFT committee member. Otherwise, if $P_i \neq Q_j$, P_i decides whether it is a potential proposer or a BFT committee member using the following conditions:

- i. P_i checks whether the condition (??) is satisfied:

$$\frac{H(r, 0, \gamma_i^r, coin_r, W_r)}{\text{MaxUnit256}} > \max\{0, 1 - a_i p\} \quad (4)$$

where γ_i^r the committed random number by P_i for the height r (see Step ??), $coin_r = \text{ZeusChain}$. Note that the value of $coin_r$ could be calculated differently. For example, in the future release of ZeusChain chain, $coin_r$ will be calculated using randao BLS common coin protocol. If the condition (??) is not satisfied, P_i moves to Step ??. If the condition (??) is satisfied, then P_i is a potential proposer. P_i packages all received transactions and then generate the candidate block B_i^r as follows (see Section ??):

$$B_i^r = \{r, t_r, TX_i^r, H(B^{r-1}), P_i\}.$$

P_i then broadcast the following message $m_i^{r,0} = (B_i^r, \gamma_i^r)$ and moves to Step ??.

¹Do we still need a data strucutre to manage the oder based on startBlock field?

- ii. The number $\tau_i \in [0, a_i]$ of votes that the node P_i has in the BFT committee is calculated as follows:

$$\tau_i = \min_{\tau} \left\{ \frac{H(r, 1, \mu_i^r, \text{coin}_r, W_r)}{\text{MaxUnit256}} < \sum_{j=0}^{\tau} \binom{a_i}{j} (p')^j (1-p')^{a_i-j} \right\} \quad (5)$$

where $\mu_i^r = \gamma_i^{r'}$ is calculated as follows: If P_i committed the random number $\gamma_i^{r'}$ for the current height interval $r \in [r' + 1, r' + \kappa']$, then $\mu_i^r = \gamma_i^{r'}$.

The alternative algorithm for the Formula (??): Let

$$\tau_i = \|\{j \in [1, a_i] : \text{the condition (??) holds}\}\|. \quad (6)$$

where

$$\frac{H(j, r, 1, \mu_i^r, \text{coin}_r, W_r)}{\text{MaxUnit256}} < p' \quad (7)$$

If $\tau_i = 0$, P_i waits for the BFT committee to confirm the block B^r . After it receives the confirmed block B^r , it moves to Step 1 of height $r + 1$. Otherwise, if $\tau_i > 0$, P_i carries out the following steps:

- A. P_i arranges all BFT committee members in the following order:

$$\text{BFT}_r = (P_{j_1}, P_{j_2}, \dots, P_{j_h}) \quad (8)$$

where $H(P_{j_1}, \mu_{j_1}^r, \text{coin}_r, W_r) < H(P_{j_2}, \mu_{j_2}^r, \text{coin}_r, W_r) < \dots < H(P_{j_h}, \mu_{j_h}^r, \text{coin}_r, W_r)$ where P_{j_s} appears τ_{j_s} times in BFT_r of (??). **Note:** BFT_r could also be ordered as in (??). But all participants must use the same ordering method. The τ_j votes for P_j could be denoted as $\langle P_j, 1 \rangle, \dots, \langle P_j, \tau_j \rangle$. Then all voting right could be ordered as

$$\text{BFT}_r = (\langle P_{j_1}, i_{j_1} \rangle, \langle P_{j_1}, i_{j_2} \rangle, \dots, \langle P_{j_h}, i_{j_h} \rangle) \quad (9)$$

where $H(P_{j_1}, i_{j_1}, \mu_{j_1}^r, \text{coin}_r, W_r) < H(P_{j_2}, i_{j_2}, \mu_{j_2}^r, \text{coin}_r, W_r) < \dots < H(P_{j_h}, i_{j_h}, \mu_{j_h}^r, \text{coin}_r, W_r)$.

- B. P_i collects and verifies all candidate blocks broadcast by all potential proposers of height r . If P_i does not receive any proposed candidate block within a give time limit², P_i should construct a default block B_i^r . A default block can be constructed using the same algorithm with the following exceptions:
- A default block contains zero transactions.
 - The miner of the default block is “NULL”.
- C. P_i orders all received candidate blocks using the following approach (note that if P_i receives multiple candidate blocks from a same potential proposer, P_i will use the first received block. Furthermore, if a potential proposer generates two candidate blocks for the same height, its staking should be confiscated in the next block). the blocks generated by P_{j_1} is not smaller than the candidate blocks generated by P_{j_2} if the following conditions are satisfied.

$$H(r, 0, \gamma_{j_1}^r, \text{coin}_r, W_r) \geq H(r, 0, \gamma_{j_2}^r, \text{coin}_r, W_r) \quad (10)$$

If the equation in (??) holds, then one can compare the hash value of the proposer's public key to determine the order of the candidate blocks. Furthermore, the default candidate block is smaller than all regular candidate blocks. For two default candidate blocks, their order is determined by their hash values. In the implementation, the node P_i has a local variable to hold the maximum candidate block that it has learned.

²This time limit should be determined by some experiments on the network delay. This time limit should be at least the time for a node to construct the block plus network delay. Furthermore, to avoid the case that some potential proposers first broadcast an empty block with zero transactions, and then broadcast some blocks with non-empty transactions, a node should wait for a sufficient time for that most proposed blocks can arrive.

- D. Broadcast the round-change message and enters BDLS BFT module (in the BDLS BFT protocol, P_i has τ_i votes. In the BDLS BFT module, the node P_i should always try to collect candidate blocks generated by other potential proposers and update its local variable for the candidate block. In the implementation, please pay attention to the synchronization issue. That is, when to broadcast the round-change message. We should have a larger E_2 value so that we have enough BFT committee members.
- E. After finishing the BDLS module, P_i should move to Step 1 of the height $r + 1$.

3. P_i carries out the following steps:

- (a) P_i wants to stake for heights $r' = r + \kappa' > r + \kappa$: P_i first selects a parameter $\kappa'' > 0$ ($\kappa'' > 0$ can be an external parameter). This means that P_i wants to participate in the blockchain governance from heights $r + \kappa' + 1$ to $r + \kappa' + \kappa''$. P_i calculates a 256-bit random number $\gamma_i = H(H(x + r)G)$ where x is the private key for P_i . P_i then calculates a series of $\kappa'' + 1$ random numbers. $\gamma_i^{r+\kappa'} = H(\gamma_i^{r+\kappa'+1})$, $\gamma_i^{r+\kappa'+1} = H(\gamma_i^{r+\kappa'+2})$, $\dots$, $\gamma_i^{r+\kappa'+\kappa''} = H(\gamma_i)$. P_i submits the following transaction to the blockchain (we hope that it will be included in blocks B^r or B^{r+1}):

$$SIG_{pk_i} \left(P_i, 0x00, \gamma_i^{r+\kappa'}, m_i, \gamma_i^{r+\kappa'}, r + \kappa', \kappa'' \right) \quad (11)$$

where $m_i = a_i \alpha > 0$ is the number of ZeusChain tokens that P_i submits for staking.

Remark: In the transaction (??), if the node P_i has more than m_i ZeusChain tokens, then the transaction is legal. If P_i currently does not have enough ZeusChain tokens, but it will have enough tokens after redeeming staked tokens at block $B^{r+\kappa'}$, then the transaction (??) is still valid. But we have to take care the scenario that P_i violates the policy and its token is confiscated later. Furthermore, the transaction (??) should only be included in the block B^r or B^{r+1} . It is illegal to include the transaction (??) in a block $B^{r'}$ with $r' > r + 1$.

- (b) P_i wants to redeem a staking: if P_i wants to redeem the staked token with a valid transaction (refcommitfor), then P_i must submit the following transaction to the blockchain:

$$SIG_{pk_i} \left(P_i, 0xFF, \gamma, m_i, \gamma_i^{r+\kappa'}, r + \kappa', \kappa'' \right) \quad (12)$$

- (c) moves to Step ??

4.8 Construct a candidate block

Assume that the node P_i collected the following regular transactions: $T_1, \dots, T_a$, staking transactions $S_1, \dots, S_b$, and redeeming transactions $R_1, \dots, R_c$, the node P_i constructs the candidate block B^r as follows:

1. verified that the block B^{r-1} is valid. This should include verify the voting results from the BFT committee, construct the BFT decision hash, and fill BFT decision hash into B^{r-1} 's header. Furthermore, it will generate a transaction to distribute the following ZeusChain tokens to B^{r-1} 's valid voting members. Note that the BFT committee members Q_2, Q_3, Q_4, Q_5 should not receive any token distribution. On the other hand, if the BDLS BFT committee only contains the members Q_2, Q_3, Q_4, Q_5 , then the transaction fee in the block B^{r-1} should be sent to NULL address.
 - (a) β_2 ZeusChain tokens
 - (b) $1 - \xi$ portion of all transaction fees within the block B^{r-1}
 - (c) all staking transaction fees contained in the block B^{r-1}
2. If a potential proposed broadcast two candidate blocks for height $r - 1$, this potential proposer's staking tokens should be transferred to P_i 's account.

3. for each transaction T_j , P_i uses Ethereum’s algorithm to check whether the transaction is valid. For each valid transaction, P_i returns the remaining gas and record the transaction. If a transaction does not have enough gas, confiscate the gas and mark it as an insufficient gas transaction. P_i allocates ξ portion of all collected transaction fee and β_1 ZeusChain tokens to itself. If $P_i = Q_1$, P_i allocates 0 tokens to itself and sends ξ portion of the transaction fees to NULL. P_i updates the State Root, Transaction Root, and Receipts Root.
4. Check ASTX to identify all staking transactions that expires in the block B^{r-1} . Remove all these expiring stakings and return all staked ZeusChain tokens to the staker.
5. For each staking transaction $S_j = SIG_{pk_j} \left(P_j, 0x00, \gamma, m_j, \gamma_i^{r+\kappa'}, r + \kappa', \kappa'' \right)$, P_i verifies whether P_j has m_j ZeusChain tokens within its account. P_i deducts m_j ZeusChain tokens from P_j ’s account and add this transaction to the World state. If P_j does not have m_j tokens in its account, P_i checks whether at the height $r + \kappa'$, P_j has sufficient tokens (we have to be careful here. P_j may spend all these redeemed tokens before height $r + \kappa'$. Thus it may be better that we do not allow this).
6. For each redeeming transaction $R_j = SIG_{pk_j} \left(P_j, 0xFF, \gamma, m_j, \gamma_i^{r+\kappa'}, r + \kappa', \kappa'' \right)$, P_i checks whether R_j is within the current World state. If yes, removes R_j from the World state and returns the related ZeusChain tokens to P_j . If R_j is not in the World state, confiscate $\min\{\gamma', P_j$ ’s account balance $\}$ tokens from P_j ’s account and transfer them to P_i .

4.9 Algorithm to calculate the total number Ω_r of staked ZeusChain tokens

ASTX is ordered using the staking transaction expiration time. Thus, one can scan all staking transactions to calculate the number Ω_r efficiently.

4.10 Security discussions

The ZeusChain consensus process is based on the Byzantine Fault Tolerance protocol BDLS for partial synchronous networks. The BFT protocol requires at least 2/3 of honest members to participate. In case that more than 1/3 members are malicious or non-participating in the process, the block chain will not achieve liveness. Ethereum 2.0 address this challenge by slashing non-participating members’ staked assets. This may introduce other challenges since, theoretically, it needs another consensus protocol to determine who is non-participating and how many members are non-participating. The ZeusChain addresses this challenge by introducing five default accounts that serves as back-up block proposers and BFT committee members.

5 Non-interactive membership/knowledge proof systems

ZeusChain is a Proof of Stake (PoS) blockchain based on the efficient BDLS Byzantine Fault Tolerance protocol [?, ?]. The algorithmic details for the ZeusChain blockchain architecture/design/implementation are described in Section ???. In this section, we describe the hub protocols for the ZeusChain to interact with other chains such as Ethereum. In order for a blockchain $\mathcal{C}$ (e.g., Ethereum) to communicate with the ZeusChain, a root smart contract needs to be deployed on the chain $\mathcal{C}$ and the chain $\mathcal{C}$ needs to register its intended digital assets on the ZeusChain. For the transaction rollups between chains, we leverage the (non-)interactive proof introduced by Goldwasser, Micali, and Rackoff [?].

5.1 Interactive membership proof systems

In a k -round interactive proof, a probabilistic polynomial-time verifier exchanges k messages with an untrusted resource-unbounded prover in order to ascertain whether a given statement is true or not. The verifier accepts the proof if it is “convinced” that the statement is true. A k -round interactive proof system has two requirements:

- **Completeness:** If the statement is true, then the verifier can be convinced of this fact by a prover.
- **Soundness:** If the statement is false, then no untrusted prover can convince the honest verifier that it is true, except with some small probability.

Shamir [?] showed that interactive proofs correspond to languages decidable in polynomial space: $\text{IP}[\text{poly}(n)] = \text{PSPACE}$. The interactive proof system is useful for proving membership problems (e.g., a given number is the multiplication of two primes, a given number is a quadratic residue, etc.). The interactive membership proof system can be extended to zero-knowledge interactive membership proof system by further requiring that

- **Zero-Knowledge:** an untrusted verifier learns nothing else other than the membership of the given input except with some small probability.

If a given input is a member of the language, then the verifier could be convinced of this fact by a prover. However, after the interactive proof protocol, the zero-knowledge property requires that the verifier could not convince anyone else of this fact using what he has learnt during the protocol run.

5.2 Interactive knowledge proof systems

In many applications, the verifier also needs to know that the prover actually owns some secret. This requirement can be formulated using Interactive Knowledge Proof systems as

- **Completeness:** If the prover knows a secret s , then the verifier is convinced of this fact by the prover.
- **Knowledge soundness:** If the prover successfully convinces the verifier, then the prover must know the secret s except with some small probability.

For the soundness proof of an interactive knowledge proof system, one constructs a knowledge extractor that interacts with the prover to extract the prover's original secret where rewinding in the protocol is allowed. We use Schnorr identification scheme as an example to show how to construct a knowledge extractor. Let g be a generator for a cyclic group of prime order q and $s < q$ be prover's secret. The public key for the prover is g^s . The Schnorr identification scheme works as follows where the input for the prover is (g, s) and the input for the verifier is (g, g^s) :

prover	$\xrightarrow{g^r}$	verifier
prover	$\xleftarrow{c}$	verifier
prover	$\xrightarrow{z=r+c \cdot s}$	verifier

The verifier accepts the proof if $g^z = g^r \cdot (g^s)^c$. The knowledge extractor uses “rewinding” to obtain two different responses from the prover for the same commitment g^r . That is,

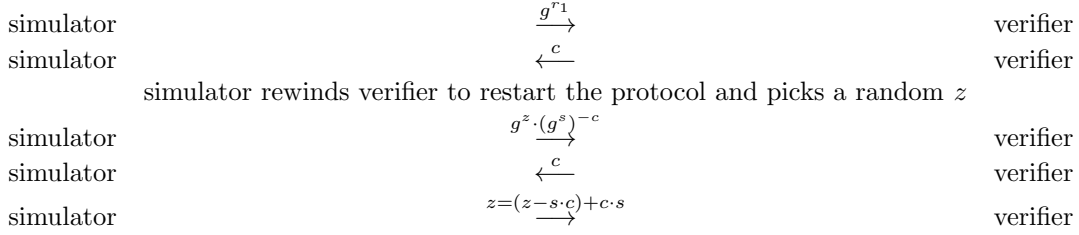
prover	$\xrightarrow{g^r}$	extractor
prover	$\xleftarrow{c_1}$	extractor
prover	$\xrightarrow{z_1=r+c_1 \cdot s}$	extractor
extractor rewinds to the previous step		
prover	$\xleftarrow{c_2}$	extractor
prover	$\xrightarrow{z_2=r+c_2 \cdot s}$	extractor

Armed with c_1, c_2, z_1, z_2 , the extractor can recover the secret $s = \frac{z_1 - z_2}{c_1 - c_2}$.

The interactive knowledge proof system can be extended to zero-knowledge interactive proof system by further requiring that

- **Zero-Knowledge:** an untrusted verifier learns zero information about the prover's secret s except with some small probability.

There are various zero-knowledge requirements such as perfect zero-knowledge, statistical zero-knowledge, and computational zero-knowledge. We will not discuss the differences of these zero-knowledge proof systems. In order to prove that an interactive knowledge proof system achieves zero knowledge, one shows that every verifier has some simulator that, without access to the prover, can produce a transcript $T = (R, c, z)$ with $g^z = R \cdot y^c$ that (perfectly, statistically, or computationally) “looks like” an interaction between the honest prover and the verifier. The simulator is normally constructed using the rewinding techniques. Schnorr identification scheme cannot be proved to be zero-knowledge if the verifier is not honest. On the other hand, it can be proved to be honest-verifier zero-knowledge by requiring that the verifier only submit challenges c using a random generator (instead of choosing c in his own way—then the distribution of c would not be random), the simulator is constructed as follows:



5.3 Non-interactive proof systems

Up to now, we have presented definitions and examples of interactive proof systems. For our design of ZeusChain, we need non-interactive proof systems. Blum, Feldman, and Micali [?] showed that a common reference string shared between the prover and the verifier is sufficient to achieve computational non-interactive zero-knowledge proofs. In practice, one uses Fiat-Shamir transform to obtain non-interactive zero-knowledge proofs in the random oracle model. For example, Schnorr identification scheme could be converted to a non-interactive protocol by picking the challenge c as $H(g, g^s, g^r, seed)$ where $seed$ is an arbitrary string.

6 Non-interactive proof systems for transaction rollups

In Section ??, we mentioned that non-interactive Zero-Knowledge proof (NIZKP) systems such as ZK-STARK/ZK-SNARK/Bullet-proof have been used for Ethereum layer 2 chains to record transactions on the main chain. It should be noted that the “zero-knowledge” aspect of these NIZKP protocols is not needed for these rollup applications and only the “succinct” aspect of the protocols plays an essential role in these applications. Indeed, in these Ethereum layer 2 rollup applications, the NIZKP protocols are only used to succinctly prove that all recorded transactions have been authorized by the involved accounts which do not need to be kept secret. It is very expensive to integrate the “zero-knowledge” function into an interactive proof system. For example, the current implementation shows that it may take more than 20 minutes to produce one single ZK-STARK proof on a powerful computer. In order to make these rollup operations practical, we need to design efficient succinct non-interactive membership proof (SNMP) systems or succinct non-interactive knowledge proof (SNKP) systems without zero-knowledge requirement. It should be noted that our SNMP/SNKP terms are essentially equivalent to the well studied Succinct Non-interactive ARGument of Knowledge (SNARK). We use SNMP to emphasize that we want to achieve membership proof. Cryptographic accumulators [?] and vector commitments (VC) can normally be used to achieve these goals. In this section, we compare Merkle tree, Diffie-Hellman, and RSA based SNMP systems that could be used for our ZeusChain design. From a high level, Merkle tree-based SNMPs are efficient in computation while they require logarithmic size openings. For Diffie-Hellman and RSA based SNMPs, the prover can open a subset of committed transactions using a single short proof. Finally Diffie-Hellman based SNMPs require large public parameter sets though RSA based SNMPs have a “constant” size public parameter set (by computing certain parameters on the fly).

6.1 Merkle tree based SNMP

Probabilistically checkable proofs (PCP) are normally used to construct succinct non-interactive arguments where the prover commits to a long PCP using a Merkle tree. A random oracle is then used to generate a few random query positions for which the prover verifiably opens the proof by providing Merkle authenticating paths. Based on PCP techniques, Micali et al [?] introduced the compact certificates of collective knowledge. Our Merkle tree based SNMP system is similar to the compact certificates in [?]. However, there is a little difference between our SNMP and the compact certificates in [?]. The compact certificates in [?] is a non-interactive knowledge proof system. Instead of using knowledge proof systems, our Merkle tree based SNMP is based on the membership proof systems. A membership proof system is sufficient for rollup applications since all we need to prove is that the batched transaction string belongs to (i.e., a member of) the set of valid transaction batches.

If we want to combine transactions $(T_1, sig_1), \dots, (T_m, sig_m)$ into one rollup batch, we can record $(T_1, \dots, T_m; \text{SNMPproof})$ on the main chain where **SNMPproof** is constructed as follows.

- For each $i \leq m$, verify that sig_i is a valid signature for the transaction T_i .
- For each $i \leq m$, calculate the total amount of tokens ω_i transferred in the transaction T_i . Let $\Omega = \omega_1 + \dots + \omega_m$.
- For each $i \leq m$, define $L_i = (T_i, sig_i, X_i, \omega_i)$ by letting $X_1 = 0$, and $X_i = \sum_{j=1}^{i-1} \omega_j$ for $i > 1$.
- Compute **root** as the Merkle tree root where the Merkle tree leaves are $L_1, \dots, L_m$.
- Define an efficient computable function Λ that maps an integer $coin \leq \Omega$ to an integer $i = \Lambda(coin)$ such that $X_i \leq coin \leq X_i + \omega_i$ where $L_i = (T_i, sig_i, X_i, \omega_i)$. The function Λ could be implemented via a binary search on the X_i values of L_i (similar to [?]).
- Let γ be the number of paths to be revealed where the value of γ is selected based on the security parameter and **COIN** be a pre-determined binary string.
- For each $j = 1, \dots, \gamma$, let
 - $coin_j = H(\text{COIN}, \text{root}, \Omega, T_1, \dots, T_m)$ where H is a hash function.
 - π_j be the Merkle tree authenticating path to the leaf $L_{\Lambda(coin_j)}$.
- The **SNMPproof** consists of **root**, Ω , Λ , $\pi_1, \dots, \pi_\lambda$ where Λ only needs to specify the λ entries that are needed in the proof.

The completeness of the protocol is straightforward. For the soundness, it is noted that, for any collection of invalid transactions with a total weight of ω , the probability that one of these invalid transactions is not selected in the **SNMPproof** is at most $(1 - \frac{\omega}{\Omega})^\lambda$. Thus if the security parameters are selected appropriately, then the protocol is sound.

In practice, the binary string **COIN** could be predetermined or could be determined dynamically. For example, if the batched transactions are recorded in the main chain block B_i , we may set $\text{COIN} = H(B_i, \dots, B_{i+t})$ for some $t \geq 0$. By requiring that the **COIN** value not to be determined before the commitment of the Merkle tree root, the attacks can be reduced significantly. This could be easily implemented since the rollup process is normally handled by a smart contract on the main chain and the batched transaction is not considered as final until the corresponding **SNMPproof** variable is assigned an actual value in a later block. On the other hand, the mechanism of dynamic value assignment for the string **COIN** makes the protocol no longer a pure non-interactive protocol. It can be considered as an interactive membership proof system.

6.2 Diffie-Hellman and RSA based SNMP

In order to open one committed leaf in the Merkle tree based SNMP protocol, the prover needs to reveal the corresponding authenticating path which is logarithm in the committed transaction size. To reduce the proof size, one may use Diffie-Hellman or RSA based sub-vector commitment schemes. In these sub-vector

commitment schemes, one only needs to construct a single group element to prove that a sub-vector is committed correctly. We first describe one Diffie-Hellman based and one RSA based sub-vector commitment schemes. Then we show how to use them for our ZeusChain rollup applications.

6.2.1 Diffie-Hellman based sub-vector commitment

Concise sub-vector commitments allow a prover to commit to a vector $\mathbf{x}$ of m messages, such that it can later open the commitment at any positioned subvector of the vector. A lot of research has been done in the last decade. The examples include mercurial vector commitments [?], vector commitments [?], sub-vector commitments [?], Pointproofs [?], and many others. Several of these schemes are sufficient for our applicaiton. In the following, we review the Diffie-Hellman based sub-vector commitment introduced by Libert and Yung [?] and Gorbunov et al [?]. The commitment scheme itself is due to [?] and the sub-vector commitment part is due to [?].

Let $e : \mathcal{G}_1 \times \mathcal{G}_2 \rightarrow \mathcal{G}_T$ be a bilinear paring function defined over groups $(\mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_T)$ of prime order p . Let $g_1, g_2, g_T = e(g_1, g_2)$ be generators for $(\mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_T)$ respectively. In order to commit vectors of size m , we need to set up public parameters as follows:

- Select a secret value $\alpha \leq p$ and compute the $2m - 1$ values in $\mathcal{G}_1$, m values in $\mathcal{G}_2$ and one value in $\mathcal{G}_T$ as follows. After the calculation of these public parameters, the value α should be securely destroyed.

$$\begin{aligned} h_1, \dots, h_m, h_{m+2}, \dots, h_{2m} \quad \text{where} \quad h_i &= g_1^{\alpha^i} \\ \bar{h}_1, \dots, \bar{h}_m \quad \text{where} \quad \bar{h}_i &= g_2^{\alpha^i} \\ h_T &= g_T^{\alpha^{m+1}} \end{aligned}$$

The commitment for a vector $\mathbf{x} = (x_1, \dots, x_m)$ is computed as

$$C(\mathbf{x}) = \prod_{i=1}^m h_i^{x_i}.$$

To open a single value x_i , the prover computes a proof

$$\pi_i = \prod_{j \neq i} h_{m+1-i+j}^{x_j} = \left(\frac{C(\mathbf{x})}{h_i^{x_i}} \right)^{\alpha^{m+1-i}}$$

The verifier accepts the proof (x_i, π_i) if and only if

$$e(C(\mathbf{x}), \bar{h}_{m+1-i}) = e\left(C(\mathbf{x}), g_2^{\alpha^{m+1-i}}\right) = e(\pi_i, g_2) \cdot g_T^{\alpha^{m+1}x_i} = e(\pi_i, g_2) \cdot h_T^{x_i}$$

To open a sub-vector $\{x_i : i \in S\}$ for a given set S , the prover computes $t_i = H(i, C(\mathbf{x}), S, \mathbf{x}[S])$ for $i \in S$ and outputs the proof

$$\pi_S = \prod_{i \in S} \pi_i^{t_i}.$$

The verifier accepts the proof $(\{x_i : i \in S\}, \pi_S)$ if and only if the following equation holds.

$$e\left(C(\mathbf{x}), \prod_{i \in S} \bar{h}_{m+1-i}^{t_i}\right) = \prod_{i \in S} e(C(\mathbf{x}), \bar{h}_{m+1-i})^{t_i} = \prod_{i \in S} e(\pi_i^{t_i}, g_2) \cdot h_T^{t_i x_i} = e(\pi_S, g_2) \cdot h_T^{\sum_{i \in S} t_i x_i}$$

6.2.2 RSA based sub-vector commitment

The limitation of the Diffie-Hellman based sub-vector commitment scheme in the preceding section is that the public parameters consist of $3m$ field elements which is not constant. In this section, we review an RSA based sub-vector commitment scheme from [?]. This scheme was adapted from [?, ?]. In order to commit vectors of size m , we need to set up public parameters as follows.

- Let $p_1, \dots, p_m$ be m distinct prime numbers. The public parameters for the scheme is a tuple $(\mathcal{G}, g, U_m)$ where $\mathcal{G}$ is a hidden order group, g is a generator of $\mathcal{G}$, and $U_m = g^{\prod_{i=1}^m p_i}$. An example of a hidden order group is the RSA group with the order $(p-1)(q-1)$.

The commitment for a vector $\mathbf{x} = (x_1, \dots, x_m)$ is computed as

$$C(\mathbf{x}) = \prod_{i=1}^m g^{x_i \prod_{j=1, j \neq i}^m p_j}$$

To open a sub-vector $\{x_i : i \in S\}$ for a given set S , the prover computes the proof $\pi_S = (\Lambda_S, \Gamma_S)$ where

$$\Lambda_S = g^{\prod_{i \notin S} p_i} \text{ and } \Gamma_S = \prod_{i=1, i \notin S}^m \left(g^{\prod_{j \notin (S \cup \{i\})} p_j} \right)^{x_i} = \left(\prod_{i=1, i \notin S}^m g^{x_i \prod_{j=1, j \neq i}^m p_j} \right)^{1 / \prod_{i \in S} p_i}$$

The verifier accepts the proof $\pi_S = (\Lambda_S, \Gamma_S)$ if and only if

$$U_m = \Lambda_S^{\prod_{i \in S} p_i} \text{ and } C(\mathbf{x}) = \Gamma_S^{\prod_{i \in S} p_i} \prod_{i \in S} U_m^{x_i / p_i} = \Gamma_S^{\prod_{i \in S} p_i} \prod_{i \in S} \left(\Lambda_S^{\prod_{j \in S \setminus \{i\}} p_j} \right)^{x_i}$$

In the above mentioned RSA based scheme, the public parameters are of constant size if we assume that the distinct prime numbers $p_1, \dots, p_m$ are computed on the fly by both the prover and the verifier. In certain applications, it may be more expensive to compute these distinct prime numbers on the fly than to keep them as part of the public parameters. Thus, the advantages of the RSA based sub-vector commitment schemes may be limited compared with Diffie-Hellman based schemes in these applications.

6.2.3 From sub-vector commitments to SNMPproofs

In the preceding sections, we reviewed Diffie-Hellman based and RSA based sub-vector commitment schemes. These schemes could be used to construct **SNMPproofs** for ZeusChain transaction rollups in the same way as in the Merkle tree based SNMP schemes. As an example, we show how to combine transactions $(T_1, sig_1), \dots, (T_m, sig_m)$ into one rollup batch $(T_1, \dots, T_m; \text{SNMPproof})$.

- For each $i \leq m$, verify that sig_i is a valid signature for the transaction T_i .
- For each $i \leq m$, calculate the total amount of tokens ω_i transferred in the transaction T_i . Let $\Omega = \omega_1 + \dots + \omega_m$.
- For each $i \leq m$, define $L_i = (T_i, sig_i, X_i, Y_i)$ by letting $X_1 = 0$, $Y_1 = \omega_1$, and $X_i = \sum_{j=1}^{i-1} \omega_j$, $Y_i = \sum_{j=1}^i \omega_j$ for $i > 1$.
- Compute the sub-vector commitment $C(\mathbf{x})$ of $\mathbf{x} = (H(T_1, sig_1), \dots, H(T_m, sig_m))$
- Define an efficient computable function Λ that maps an integer $coin \leq \Omega$ to an integer $i = \Lambda(coin)$ such that $X_i \leq coin \leq Y_i$ where $L_i = (T_i, sig_i, X_i, Y_i)$. The function Λ could be implemented via a binary search on the X_i values of L_i (similar to [?]).
- Let γ be the number of paths to be revealed where the value of γ is selected based on the security parameter and **COIN** be a pre-determined binary string.
- For each $j = 1, \dots, \gamma$, let $coin_j = H(\text{COIN}, \text{root}, \Omega, T_1, \dots, T_m)$ where H is a hash function.
- Let $S = \{\Lambda(coin_j) : j = 1, \dots, \gamma\}$ and π_S be the proof for opening the sub-vector $\{x_i : i \in S\}$.
- The **SNMPproof** consists of " $C(\mathbf{x}), \Omega, \Lambda, \pi_S, \{(T_i, sig_i) : i \in S\}$ " where Λ only needs to specify the λ entries that are needed in the proof.

7 ZeusChain with SNMPproofs: inter-blockchain communications and digital asset swapping

ZeusChain is a public blockchain with Proof of Stake (PoS) consensus (see Section ?? for details). The block proposers and validators are selected based on the amount of staked assets and an unpredictable random beacon. The identity of the proposers and validators are unpredictable before the block is proposed and validated (see Section ?? for details). There are essential differences between ZeusChain and other solutions on the market. As we have mentioned, validators on Ethereum 2.0 shard chains are randomly assigned by the beacon chain. This may lead to DoS or bribing attacks against the shard chains. To increase performance, Polygon blockchain uses a block producer (which is selected by stakers and serves as the block producer for a checkpoint period) to produce blocks and a small number of validators to validate blocks and rollup checkpoint data to the Ethereum chain. This opens a door for one to bribe these small number of validators and block proposers (e.g., the block proposer may *exclude* or *delay* certain transactions on purpose). The ZeusChain PoS blockchain architecture and mechanisms are described in Section ?. In order for the ZeusChain to roll up certain data to the other chains, these data must be finalized on the ZeusChain blocks. The ZeusChain consists of several co-chains that serve for different purposes. In the version 1.0 of ZeusChain, we will implement layer 2 functions and digital asset swapping tools. The layer 2 functions are the basis for the asset swapping functions.

7.1 ZeusChain chains as Layer 2 chains

Some chains of ZeusChain could be used as layer 2 chains for other chains such as Ethereum blockchain. It should be noted that Ethereum Layer 2 techniques such as Rollup/Plasma/Validium do not require a separate blockchain. In most Layer 2 implementations, the relayer or aggregator has the full power to determine which transaction to include and how to order these transactions in the rollup package. The advantages for this kind of implementation is that the transaction per second could be scaled up to tens of thousands which is normally determined by the computing capability of the relayer (aggregator). One of the disadvantages is that the relayer (aggregator) could be a single point of failure for the implementation. In some implementations, any user can serve as the relayer (aggregator) if he stakes some assets in the main chain layer 2 smart contract). For example, the relayer (aggregator) may re-order the transactions/exclude certain transactions/insert new transactions to maximize his reward via Miner Extractable Value (MEV). In our approach, we use a trusted ZeusChain chain as a layer 2 for other chains. The advantages of our approach is that the transaction ordering is no longer dependent on a single user and the front runner attacks could be reduced significantly. The disadvantage of our approach is that there is a limit on the tps (transaction per second) scalability. That is, the range of scalability depends on ZeusChain chain's transaction per second.

ZeusChain supports three kinds of Inter-Blockchain Communications: transaction rollup, checkpoint rollup, and account update. These approaches correspond to ZK-Rollup, Plasma, and Validium approaches in Ethereum layer 2 techniques. In order for a ZeusChain $\mathcal{C}$ to serve as a layer 2 chain for another chain $\mathcal{C}_0$, we need to deploy a root contract Ξ on the chain $\mathcal{C}_0$ which maintains

- an updated list **GOV** (where members are identified using their public keys) of $\mathcal{C}$'s governance structure for the purpose of layer 2 chain, and
- an updated list **ASSETS** of digital assets owned by users that could be transacted on chain $\mathcal{C}$.

To save bandwidth, **GOV** may be committed as a Merkle tree root in the smart contract. Initially, when a contract Ξ is deployed on $\mathcal{C}_0$, both lists are initialized as "genesis status" of the contract. Over time, both **GOV** and **ASSETS** are kept up-to-date by $\mathcal{C}$ by posting in $\mathcal{C}_0$ suitable transactions signed by one of $\mathcal{C}$'s latest validator set contained in the **GOV**. The governance structure **GOV** may or may not be the same as $\mathcal{C}$'s consensus protocol governance structure. For most applications, **GOV** is independent of $\mathcal{C}$'s consensus protocol. For example, **GOV** may be a set of relayers (aggregators) who deposit certain amount of stakes in the smart contract Ξ , and **GOV**'s role is to record the finalized blocks to the chain $\mathcal{C}_0$. The governance structure **GOV** could also be a more complicated access structure. For example, **GOV** could be defined as a role based access control (RBAC) that are commonly used in medical data and educational data processing environments or a rule-based access

control (RAC) that are commonly used to enforce certain security policies. Other commonly used access control models include attribute-based access control (ABAC), discretionary access control (DAC), history based access control (HBAC), mandatory access control (MAC), organization based access control (OrBAC), and generalized monotone access controls. In some applications, the general adversary structures models are often used. That is, the set of validator nodes must not be a member of the adversary structure.

In order for a user to use the layer 2 service on $\mathcal{C}$, the user needs to deposit certain amount of $\mathcal{C}_0$ digital asset in the root contract account Ξ on $\mathcal{C}_0$. In order for the user to exit from the layer 2 service on $\mathcal{C}$, the user burns the digital asset in the contract Ξ by transferring his digital asset in the contract Ξ to the NULL address on the chain $\mathcal{C}$. After this burning transaction is recorded in the contract Ξ on chain $\mathcal{C}_0$, the user can transfer the digital asset to his regular account on chain $\mathcal{C}_0$.

- **transaction rollup:** This approach is similar to the ZK-rollup technique in Ethereum layer 2 techniques. Instead of using ZK-SNARK/ZK-STARK, this approach uses **SNMPproofs** to prove the validity of the transactions. The members of **GOV** periodically batch finalized transactions on $\mathcal{C}$ that are of interest to the chain $\mathcal{C}_0$ and upload the batches to the chain $\mathcal{C}_0$. Assume that members of **GOV** batch transactions related to **ASSETS** at least once per 100 finalized $\mathcal{C}$ blocks and that the last time the rollup happened is after block B_n on $\mathcal{C}$ is finalized. After the blocks $B_{n+1}, \dots, B_{n+j}$ ($j \leq 100$) on $\mathcal{C}$ are finalized, an authorized member set in the current **GOV** selects all transaction $T_1, \dots, T_t$ included in the blocks $B_{n+1}, \dots, B_{n+j}$ that are related to **ASSETS** and batch them in a single packet $(T, B_j^h, \text{SNMPproof})$ with the SNMP proof **SNMPproof** and submits it to the smart contract Ξ , where B_i^h is the header of block B_i . Though appropriate parameter selection could be used to guarantee that the security of the **SNMPproof** is at least 2^{-128} , the malicious relayer (aggregator) set may still cheat or exclude certain transactions that it should include. To further increase the security of **SNMPproof** and to defeat malicious noninclusion, after rollup process the smart contract Ξ accepts challenge from the public world that. If the challenge is successful, all staked assets by the relayer (aggregator) are slashed and awarded to the challenger.
- **checkpoint rollup:** This approach is similar to but different from the Plasma technique (such as Polygon blockchain) in Ethereum layer 2 techniques. For example, Polygon's MoreVP plasma architecture contains three layers: Ethereum smart contracts, Cosmos-SDK based Heimdall layer that communicates between Ethereum smart contracts and Polygon Bor layer, and GO-Ethereum based Bor layer that produces blocks. In the polygon approach, the block consensus is achieved at the Heimdall layer. ZeusChain itself is a fully functional secure blockchain and block finality is achieved at the first layer. The role of **GOV** members is to rollup checkpoints of ZeusChain to the other chains and communicate messages from other chains to the ZeusChain so that these messages could be included in the ZeusChain blocks.
- **account balance update:** This approach is similar to the Validium technique in Ethereum layer 2 techniques. Instead of using ZK-SNARK/ZK-STARK, this approach uses **SNMPproofs** to prove the validity of the transactions. We illustrate this approach using a simple example. The members of **GOV** periodically update the **ASSETS** data structure in the contract $\mathcal{C}_0$ directly without listing itemized transactions. Assume that **ASSETS** is updated once per 100 finalized blocks on the chain $\mathcal{C}$ and **ASSETS** was updated when the block B_n is finalized. After the blocks $B_{n+1}, \dots, B_{n+100}$ on $\mathcal{C}$ are finalized, an authorized relayer (aggregator) set in the current **GOV** calculates the updated **ASSETS** from the current **ASSETS** using all transactions in these newly finalized 100 blocks. Assume that the updated **ASSETS** could be obtained by overwriting the following set $\{(a_1, b_1), \dots, (a_t, b_t)\}$ where user a_i 's new balance is b_i for $i \leq t$. Let T_i be a list of transactions showing that a_i 's new balance is b_i . Then the authorized relayer (aggregator) set constructs a **SNMPproof** for the data sets $\{T_1, \dots, T_t, B_{n+1}^h, \dots, B_{n+100}^h, n, n+100\}$ where B_i^h is the header of block B_i . The authorized relayer (aggregator) set submits the input $\{(a_1, b_1), \dots, (a_t, b_t), n, n+100, \text{SNMPproof}\}$ to the smart contract Ξ . Though appropriate parameter selection could be used to guarantee that the security of the **SNMPproof** is at least 2^{-128} , the malicious relayer (aggregator) set may still choose to incorrectly update some user's new balance in this transaction. To further increase the security of **SNMPproof** and to defeat malicious noninclusion, after this account balance update transaction, the smart contract Ξ accepts

challenge from the public world that, based on these 100 finalized blocks, some balances are incorrect or not updated. If the challenge is successful, all staked assets by the relayer (aggregator) set are slashed and awarded to the challenger.

7.2 Cross chain AMM on ZeusChain

Lightning network [?] was the first blockchain scaling solution. It introduced the Hashed Timelock Contract (HTLC) protocol. This kind of contract was used to design cross chain atomic swapping protocols. Decred’s atomic swapping implementation could be used to swap properties on several Bitcoin-like chains. Multichain’s SMPC network uses Gennaro and Goldfeder’s Threshold Signature Scheme (TSS) [?] for signing transactions on each supported blockchain. Multichain’s solution includes a cross-chain bridge and a cross-chain router. In multichain’s bridge solution, a user can send asset on chain $\mathcal{C}_1$ to a smart contract on $\mathcal{C}_1$. The SMPC network mints corresponding 1:1 wrapped token on the target chain $\mathcal{C}_2$ for the user. In multichain’s router solution, there are multichain pools for the same token (e.g., USDC on various chains) on various chains. When a user moves a token from chain $\mathcal{C}_1$ to chain $\mathcal{C}_2$, the SMPC protocol first uses the same token held in the pool and then mints new tokens if the chain $\mathcal{C}_2$ pool does not have enough tokens. These minted tokens will be burned later when enough tokens available in the pool. The Thorchain is another cross chain asset swapping system based on Gennaro and Goldfeder’s Threshold Signature Scheme (TSS) [?]. Though TSS scheme achieves some kind of decentralization, it does not achieve full decentralization that the Byzantine Fault Tolerance based ZeusChain achieves.

In the following, we use an example to show how to establish an AMM on ZeusChain $\mathcal{C}$. Assume that there are two chains $\mathcal{C}_1$ and $\mathcal{C}_2$. T_1 is a digital token on chain $\mathcal{C}_1$ and T_2 is a digital token on chain $\mathcal{C}_2$. In order to establish an AMM for tokens T_1 and T_2 in the ZeusChain $\mathcal{C}$, we need $\mathcal{C}$ to serve as a layer 2 chain for both chains $\mathcal{C}_1$ and $\mathcal{C}_2$ as discussed in Section ???. That is, we deploy a layer 2 root smart contract Ξ_1 on $\mathcal{C}_1$ and a layer 2 root smart contract Ξ_2 on $\mathcal{C}_2$. T_1 is registered in the **ASSETS** of Ξ_1 and T_2 is registered in the **ASSETS** of Ξ_2 . A market patron can establish an AMM $C(T_1, T_2)$ on $\mathcal{C}$ by depositing certain amount of T_1 tokens in Ξ_1 and corresponding amount of T_2 tokens in Ξ_2 . A user can carry out a swapping activity using one of the following approaches:

- The user first deposits some T_1 token in the smart contract Ξ_1 . After this transaction is recorded on the chain $\mathcal{C}$, the user can use $C(T_1, T_2)$ to carry out the swapping transaction on the chain $\mathcal{C}$ directly (the same way that a user carries out a swap in Uniswap on Ethereum). After the swapping, the user can keep the received T_2 token on chain $\mathcal{C}$ or move the token to chain $\mathcal{C}_2$ using the smart contract Ξ_2 .
- The user is not aware of the existence of chain $\mathcal{C}$. The user uses the user interface to swap the token T_1 for the token T_2 in the same way that a user uses the Uniswap on Ethereum. Behind the scene, after the user initiates the swapping activity, the following steps are carried out in the background:
 - user’s T_1 token is deposited to the smart contract Ξ_1 on chain $\mathcal{C}_1$
 - Ξ_1 ’s transaction is recorded on the chain $\mathcal{C}$
 - $C(T_1, T_2)$ calculates the amount of T_2 token corresponding to these T_1 tokens and credit these T_2 token to the user’s account on $\mathcal{C}_2$ (the user is not aware of this account)
 - user’s T_2 token is transferred to the smart contract Ξ_2 on chain $\mathcal{C}_2$
 - these T_2 token in Ξ_2 is transferred to the user’s account on chain $\mathcal{C}_2$

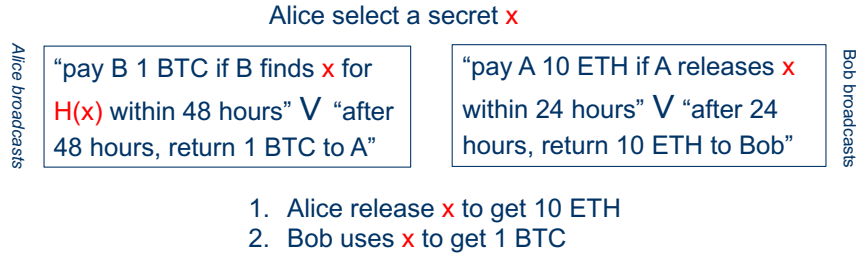
Different chains have different security strength. In the following, we list a few potential challenges in cross chain digital assets swapping based on chain security levels.

1. Chain $\mathcal{C}_1$ is less secure: In this case, the transactions on the chain $\mathcal{C}_1$ may be reversed. The adversary may swap token T_1 for T_2 using $C(T_1, T_2)$. The transactions on $\mathcal{C}_1$ is recorded on block B_{n+1} of $\mathcal{C}_1$. After the user receives token T_2 , she may reverse $\mathcal{C}_1$ to block B_n . Thus it will receive free tokens on the secure chain $\mathcal{C}_2$. There is no thing wrong with the ZeusChain here. When a patron decides to establish a market maker $C(T_1, T_2)$, the patron should be aware of this kind of risks and should take full responsibility for the potential digital assets loss under this attack.

2. Arbitraging: Assume that both $\mathcal{C}_1$ and $\mathcal{C}_2$ are secure blockchains and finalized blocks could not be reversed. The time intervals for producing blocks on the two chains are generally different and the transaction rollup from one chain to another chain takes time. This will introduce some arbitraging chances. For example, when a user tries to swap T_1 for T_2 , a front runner may observe this activity and try to make the price for token T_2 more expensive in the market maker $C(T_1, T_2)$. This challenge could be significantly reduced if the user transfer the token T_1 to the chain $\mathcal{C}$ and then carry out the swapping activity on chain $\mathcal{C}$ directly. Furthermore, the patron may use smoother Coinswap configurations (see Wang [?]) to reduce front runner attacks.

7.3 Atomic swapping on ZeusChain

The ZeusChain cross chain automated market makers require liquidity providers. ZeusChain also supports peer to peer atomic swap without liquidity providers. Traditionally, the atomic swap as shown in the following figure is based on the Hashed Timelock Contract (HTLC) protocol [?].



There are some implementation of atomic swapping on Ethereum compatible blockchains such as SAFE and NXTP. These atomic swapping systems require several round of complicated message communications across chains. ZeusChain provides a one stop peer to peer atomic swapping solution.

Assume that Alice has some token T_1 on the chain $\mathcal{C}_1$ and Bob has some token T_2 on the chain $\mathcal{C}_2$. Furthermore, assume that Alice and Bob has transferred their T_1, T_2 tokens to the ZeusChain $\mathcal{C}$ using the approaches discussed in Section ?? . ZeusChain team provides a user interface (independent of the $\mathcal{C}$) for users to post their demands and to negotiate token swapping prices. After Alice and Bob agree to swap their T_1, T_2 tokens, Alice initiates the atomic swap function on the $\mathcal{C}$ to deploy a smart contract `atomic-swap( $T_1, T_2$ )` on chain $\mathcal{C}$ and deposits her T_1 tokens into this contract. Bob can submit his T_2 tokens to this smart contract to receive the T_1 tokens and the smart contract will send the T_2 tokens to Alice.

8 Conclusion

In conclusion, ZeusChain presents a novel and robust approach to achieving secure and efficient inter-blockchain communication. By integrating a comprehensive Layer 1 blockchain and a Byzantine Fault Tolerance consensus mechanism BDLS, ZeusChain ensures the security and scalability necessary for a multi-chain future. The introduction of various rollup techniques, such as transaction rollup, checkpoint rollup, and account update, showcases the versatility of ZeusChain in handling different inter-chain communication needs.

The implementation of SNMP proofs provides a secure and efficient method for validating transactions across chains without compromising the system’s performance. Furthermore, ZeusChain’s ability to function as a Layer 2 solution for other blockchains highlights its potential to significantly enhance the scalability and interoperability of existing blockchain networks.

Through its innovative design and focus on security, ZeusChain not only addresses the current challenges faced by blockchain technology but also sets a new standard for future developments in the field. The ZeusChain ecosystem, with its stablecoin economy and integrated DeFi applications, promises to foster a new era of decentralized finance, enabling seamless and trustless transactions across diverse blockchain platforms.

References

- [1] M. Ben-Or. Another advantage of free choice: Completely asynchronous agreement protocols (extended abstract). In *Proc. 2nd ACM PODC*, pages 27–30, 1983.
- [2] J. Benaloh and M. Mare. One-way accumulators: A decentralized alternative to digital signatures. In *Workshop on the Theory and Application of Cryptographic Techniques*, pages 274–285. Springer, 1993.
- [3] M. Blum, P. Feldman, and S. Micali. Non-interactive zero-knowledge and its applications. In *20th Annual ACM Symposium on Theory of Computing*, pages 103–112. 1988.
- [4] M. Campanelli, D. Fiore, N. Greco, D. Kolonelos, and L. Nizzardo. Vector commitment techniques and applications to verifiable decentralized storage. *IACR Cryptol. ePrint Arch.*, 2020:149, 2020.
- [5] M. Castro and B. Liskov. Practical byzantine fault tolerance and proactive recovery. *ACM TOCS*, 20(4):398–461, 2002.
- [6] D. Catalano and D. Fiore. Vector commitments and their applications. In *International Workshop on Public Key Cryptography*, pages 55–72. Springer, 2013.
- [7] D. Dolev and H.R. Strong. Polynomial algorithms for multiple processor agreement. In *Proc. 14th ACM STOC*, pages 401–407. ACM, 1982.
- [8] C. Dwork, N. Lynch, and L. Stockmeyer. Consensus in the presence of partial synchrony. *JACM*, 35(2):288–323, 1988.
- [9] Ethereum. Rando: A DAO working as RNG of ethereum, 2017. <https://github.com/randao/randao> and https://www.randao.org/whitepaper/Randao_v0.85_en.pdf.
- [10] M.J. Fischer, N. A. Lynch, and M.S. Paterson. Impossibility of distributed consensus with one faulty process. *JACM*, 32(2):374–382, 1985.
- [11] R. Gennaro and S. Goldfeder. Fast multiparty threshold ecDSA with fast trustless setup. In *Proc, 2018 ACM CCS*, pages 1179–1194, 2018.
- [12] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on computing*, 18(1):186–208, 1989.
- [13] S. Gorbunov, L. Reyzin, H. Wee, and Z. Zhang. Pointproofs: Aggregating proofs for multiple vector commitments. In *Proc. ACM CCS*, pages 2007–2023, 2020.
- [14] J. Katz and C.-Y. Koo. On expected constant-round protocols for byzantine agreement. *Journal of Computer and System Sciences*, 75(2):91–112, 2009.
- [15] R. Lai and G. Malavolta. Subvector commitments with application to succinct arguments. In *Annual International Cryptology Conference*, pages 530–560. Springer, 2019.
- [16] L. Lamport, R. Shostak, and M. Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 4(3):382–401, 1982.
- [17] B. Libert and M. Yung. Concise mercurial vector commitments and independent zero-knowledge sets with short proofs. In *Theory of Cryptography Conference*, pages 499–517. Springer, 2010.
- [18] S. Micali, L. Reyzin, G. Vlachos, R.S. Wahby, and N. Zeldovich. Compact certificates of collective knowledge. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 626–641. IEEE, 2021.
- [19] M. Pease, R. Shostak, and L. Lamport. Reaching agreement in the presence of faults. *JACM*, 27(2):228–234, 1980.

- [20] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments. <https://lightning.network/lightning-network-paper.pdf>, 2016.
- [21] A. Shamir. IP=PSPACE. *Journal of the ACM (JACM)*, 39(4):869–877, 1992.
- [22] Yongge Wang. Byzantine fault tolerance in partially connected asynchronous networks. <http://eprint.iacr.org/2019/1460>, 2019.
- [23] Yongge Wang. Automated market makers for decentralized finance (defi). *arXiv preprint arXiv:2009.01676*, 2020.
- [24] Yongge Wang. The adversary capabilities in practical byzantine fault tolerance. In *Proc. 17th International Workshop on Security and Trust Management, STM 2021, LNCS 13075*, pages 1–20, 2021.
- [25] Yongge Wang. Byzantine fault tolerance for distributed ledgers revisited. *Distributed Ledger Technologies: Research and Practice*, 1(1):1–26, 2022.